

HW3 B1121123 巫煊崙

程式碼

```
'timescale 1 ns / 1ps
'define DLY #1

module FSM_CNTL ( WENAB,          // 0/P
                  RENAB,          // 0/P
                  SelfTST,
                  WRMEM_Done,
                  RDMEM_Done,
                  CLK, RESET_ );

// I/O signals
output reg WENAB, RENAB;        // Write/Read enable
input SelfTST;
input WRMEM_Done, RDMEM_Done;
input CLK, RESET_;             // clock and RESET_
// local declaration
parameter [1:0] ST0=2'b00, ST1=2'b01, ST2=2'b10, ST3=2'b11;
// Internal control signals
reg [1:0] Current_ST, Next_ST;
// ----
```

```
// my design
```

```
always @(posedge CLK or negedge RESET_) begin
    if(!RESET_) Current_ST <= ST0;
    else          Current_ST <= 'DLY Next_ST;
end
```

```
always @(*) begin
    case(Current_ST)
        ST0: Next_ST = SelfTST ? ST1 : ST0;
        ST1: Next_ST = WRMEM_Done ? ST2 : ST1;
        ST2: Next_ST = RDMEM_Done ? ST3 : ST2;
        ST3: Next_ST = ST3;
        default: Next_ST = ST0;
    endcase
end
```

```
always @(*) begin
    case(Current_ST)
        ST0: {WENAB, RENAB} = 2' b00;
        ST1: begin
            if (WRMEM_Done) {WENAB, RENAB} = 2' b11;
            else             {WENAB, RENAB} = 2' b00;
        end
    endcase
end
```

```

        end
        ST2: begin
            if (RDMEM_Done) {WENAB, RENAB} = 2' b10;
            else             {WENAB, RENAB} = 2' b11;
        end
        ST3: {WENAB, RENAB} = 2' b10;
        default: {WENAB, RENAB} = 2' b10;
    endcase
end
// ----
endmodule

```

說明

這次作業是有限狀態機 (FSM, Finite State Machine) 用來控制記憶體的讀取與寫入。

1. Module Ports

```

module FSM_CNTL ( WENAB,           // O/P 寫致能 (Active Low)
                  RENAB,           // O/P 讀致能 (Active High)
                  SelfTST,         // I/P 自我測試啟動訊號
                  WRMEM_Done,      // I/P 寫入完成訊號 (由 HW2 回傳)
                  RDMEM_Done,     // I/P 讀取完成訊號 (由 HW2 回傳)
                  CLK, RESET_ );  // I/P 時脈與非同步重設

```

SelfTST 是 ENABLE 訊號，等於 1 時電路才會開始動作。

WRMEM_Done 和 RDMEM_Done 則是來自外部（計數器數到 31 時），告訴狀態機「目前任務已完成，可以跳到 NEXT_STATE」。

WENAB 與 RENAB 用來直接控制記憶體의讀寫。

2. State Parameters

```
parameter [1:0] ST0=2'b00, ST1=2'b01, ST2=2'b10, ST3=2'b11;  
reg [1:0] Current_ST, Next_ST;
```

這裡定義了 4 個狀態

- ST0 (Idle/Reset)：閒置與等待狀態。
- ST1 (Write State)：執行 32 筆資料的寫入。
- ST2 (Read State)：執行 32 筆資料的讀取。
- ST3 (Done/End)：測試結束。

3. FSM

```
always @(posedge CLK or negedge RESET_) begin  
    if(!RESET_) Current_ST <= ST0;  
    else          Current_ST <= 'DLY Next_ST;  
end
```

```

always @(*) begin
    case(Current_ST)
        ST0: Next_ST = SelfTST ? ST1 : ST0;
        ST1: Next_ST = WRMEM_Done ? ST2 : ST1;
        ST2: Next_ST = RDMEM_Done ? ST3 : ST2;
        ST3: Next_ST = ST3;
        default: Next_ST = ST0;
    endcase
end

```

```

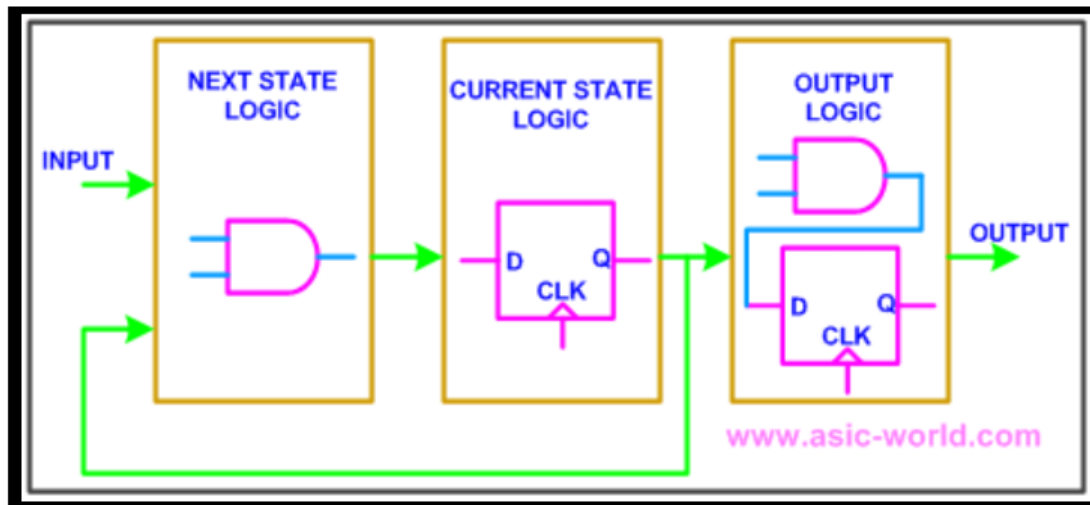
always @(*) begin
    case(Current_ST)
        ST0: {WENAB, RENAB} = 2' b00;
        ST1: begin
            if (WRMEM_Done) {WENAB, RENAB} = 2' b11;
            else {WENAB, RENAB} = 2' b00;
        end
        ST2: begin
            if (RDMEM_Done) {WENAB, RENAB} = 2' b10;
            else {WENAB, RENAB} = 2' b11;
        end
    end
end

```

```

        ST3:    {WENAB, RENAB} = 2'b10;
        default: {WENAB, RENAB} = 2'b10;
    endcase
end

```



上圖為有限狀態機（FSM）的系統方塊圖。

由圖中可知， next_state 由 Combinational Logic 計算，而 current_state 則經由 Sequential Logic 暫存器進行更新，對應的 Verilog 程式碼如上。

最後的 Output Logic，我習慣用三段式（3-always-block）的寫法，將狀態轉移與訊號輸出分開寫，這樣提升了後續 Debug 與維護的便利。

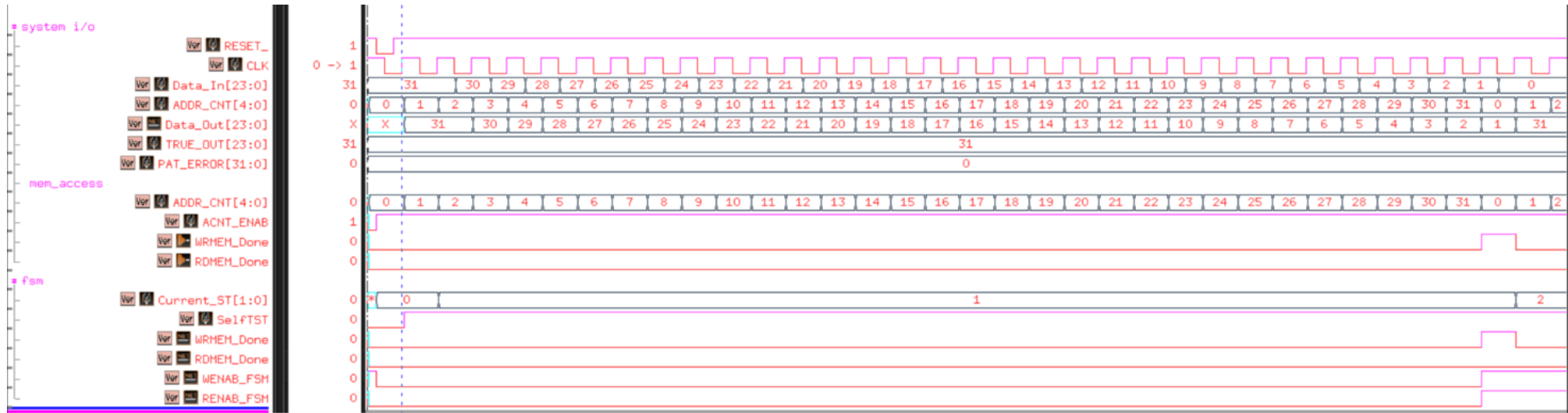
模擬結果

```
Parsing design file 'RF1_32x24b.v'
Top Level Modules:
    MEM_FSM_RTb
TimeScale is 1 ns / 1 ps
Starting vcs inline pass...

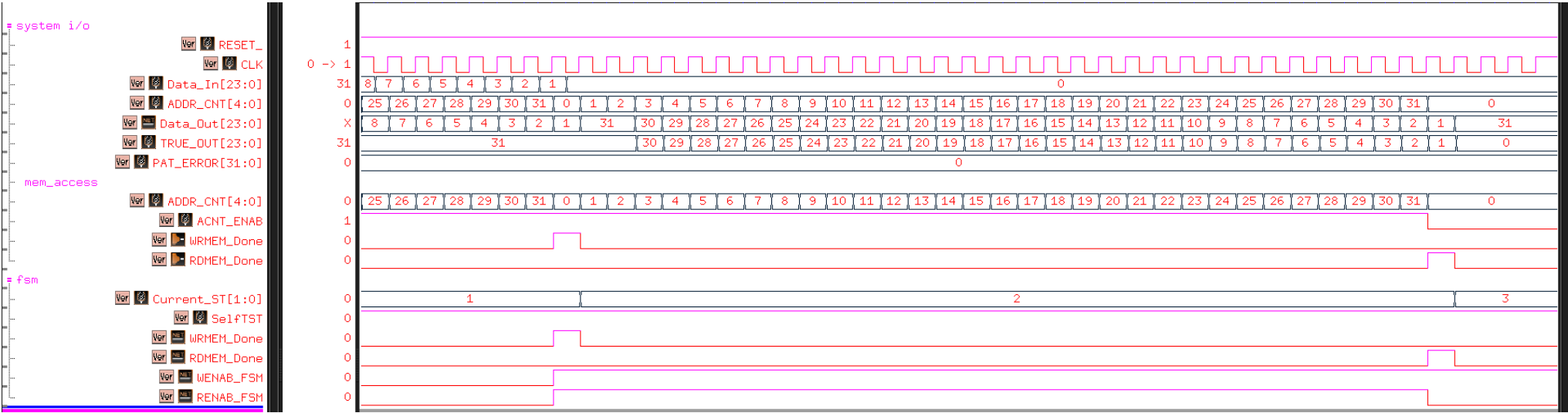
1 module and 0 UDP read.
recompiling module MEM_FSM_RTb
rm -f _cuarc*.so _csrc*.so pre_vcsobj_*.so share_vcsobj_*.so
ld -shared -Bsymbolic -o ../simv.daidir//_cuarc0.so objs/amcQw_d.o
rm -f _cuarc0.so
if [ -x ../simv ]; then chmod a-x ../simv; fi
g++ -o ../simv -rdynamic -Wl,-rpath='$_ORIGIN'/simv.daidir -Wl,-rpath=../simv.daidir -Wl,-rpath=/usr/cad/synopsys/vcs/2022.06/linux64/lib -L/usr/cad/synopsys/vc
2022.06/linux64/lib -Wl,-rpath-link=../_3124_archive_1.so_prev_archive_1.so _cuarc0.so SIM_l.o rmapats_mop.o rmapats.o rmar.o rmar_nd.o rmar_llvm_0_1.o
ar_llvm_0_0.o -lnuma -lvirsim -lerrorinf -lsnpsmalloc -lvfs -lvcsnew -lsimprofile -luclintative /usr/cad/synopsys/vcs/2022.06/linux64/lib/vcs_tls.o -Wl
whole-archive -lvcsucli -Wl,-no-whole-archive _vcs_pli_stub.o /usr/cad/synopsys/vcs/2022.06/linux64/lib/vcs_save_restore_new.o /usr/cad/synopsys/verdi/2
2.06//share/PLI/VCS/LINUX64/pli.a -ldl -lc -lm -lpthread -ldl
../simv up to date
Chronologic VCS simulator copyright 1991-2022
Contains Synopsys proprietary information.
Compiler version T-2022.06_Full64; Runtime version T-2022.06_Full64; May 16 07:50 2026
*Verdi* Loading libsscore_vcs202206.so
FSDB Dumper for VCS, Release Verdi_T-2022.06, Linux x86_64/64bit, 05/29/2022
(C) 1996 - 2022 by Synopsys, Inc.
*Verdi* FSDB WARNING: The FSDB file already exists. Overwriting the FSDB file may crash the programs that are using this file.
*Verdi* : Create FSDB file 'HW3_MEM_FSM.fsdb'
*Verdi* : Begin traversing the scope (MEM_FSM_RTb), layer (0).
*Verdi* : End of traversing.
// ----- Write operation -----
// ----- Read operation -----
// -----
// Congratulations!! Reading Operation is correct !!
// -----

$finish called from file "HW3_MEM_FSM_rtb.v", line 117.
$finish at simulation time 1362000
V C S S i m u l a t i o n R e p o r t
Time: 1362000 ps
CPU Time: 0.230 seconds; Data structure size: 0.0Mb
Sat May 16 07:50:29 2026
CPU time: .196 seconds to compile + .178 seconds to elab + .146 seconds to link + .250 seconds in simulation
[stu004@vlsi LAB3]$
```

write 波形圖



read 波形圖



在寫程式時，我注意到：在 ST0 的狀態下，{WENAB, RENAB} 應為 2' b10

但波形圖卻在 ST0 的狀態下，{WENAB, RENAB}發生了改變，這部分可能是作業有誤

Table 1. State table and state diagram

Input		Internal Input		Current State	Output	
RESET_	SelfTST	WRENAB_Done	RDENAB_Done		WENAB_FSM	RENAB_FSM
0	X	X	X	ST0	1	0
1	0	0	0	ST0	1	0
1	1	0	0	ST1	0	0
1	1	1	0	ST2	1	1
1	1	1	1	ST3	1	0

